Université des Sciences et de la Technologie MB-Oran
Faculté des mathématiques et informatique
Département d'informatique
Niveau : Master 1

Polycopié de Cours — Bases de Données Avancées

CHAPITRE 2

Programmation SQL avancée

Chapitre 2 — Programmation SQL avancée

Ce chapitre approfondit la maîtrise du langage SQL au-delà des requêtes simples : jointures et sous-requêtes complexes, fonctions analytiques, vues, procédures et fonctions stockées, déclencheurs, gestion des erreurs, ainsi qu'une ouverture vers les langages de requête propres aux bases de données NoSQL (JSONiq, Cypher), qui illustrent par contraste les choix de conception du SQL relationnel.

2.0 Données d'exemple utilisées dans ce chapitre

Les exemples des sections 2.1 à 2.2 s'appuient tous sur les trois tables suivantes, qu'il est utile de garder sous les yeux tout au long de la lecture.

Table Departement(id_dept, nom_departement)

id_dept	nom_departement
10	Ventes
20	Marketing
30	Informatique

Table Employe(id_emp, nom, email, salaire, id_dept, id_responsable, fonction)

id_emp	nom	email	salaire	id_dept	id_responsable	fonction
1001	Amel	amel@corp.com	45000	10	NULL	ADMIN
1002	Bilal	bilal@corp.com	38000	10	1001	EMPLOYE
1003	Camille	camille@corp.com	41000	10	1001	EMPLOYE
1004	Driss	driss@corp.com	36000	10	1001	EMPLOYE
1005	Emna	emna@corp.com	39000	10	1001	EMPLOYE
1006	Farid	farid@corp.com	33000	10	1001	EMPLOYE
1007	Ghali	ghali@corp.com	52000	20	NULL	MANAGER
1008	Hana	hana@corp.com	41000	20	1007	EMPLOYE
1009	Imad	imad@corp.com	60000	30	NULL	MANAGER
1010	Jana	jana@corp.com	30000	NULL	NULL	EMPLOYE

Table Projet(id_projet, id_resp, budget)

id_projet	id_resp	budget
1	1003	150000
2	1005	80000
3	1001	120000

2.1 Rappels SQL avancés : jointures, sous-requêtes, agrégation, vues, procédures

2.1.1 Les jointures

La jointure combine des tuples de plusieurs relations selon une condition de correspondance, généralement une égalité entre une clé étrangère et la clé primaire référencée. C'est l'opération qui permet de reconstituer, au moment de l'interrogation, l'information répartie entre plusieurs tables normalisées.

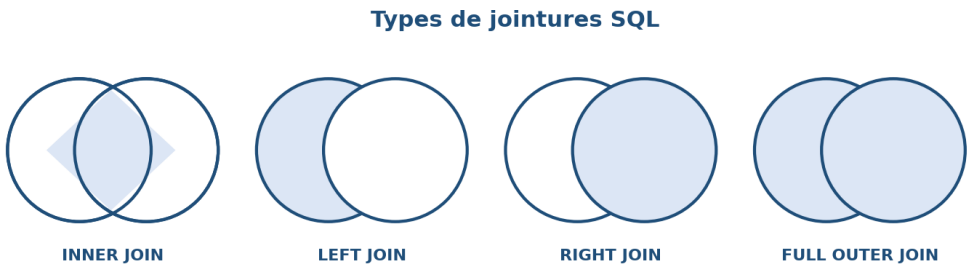


Figure 2.1 — Représentation visuelle des principaux types de jointures SQL

Chaque type de jointure définit différemment quels tuples « sans correspondance » sont conservés dans le résultat :

- **INNER JOIN (jointure interne) :** ne conserve que les tuples des deux tables qui satisfont la condition de jointure ; les tuples sans correspondance sont exclus du résultat.
- **LEFT OUTER JOIN (jointure externe gauche) :** conserve tous les tuples de la table de gauche, complétés par des valeurs NULL lorsqu'aucune correspondance n'existe à droite.
- **RIGHT OUTER JOIN (jointure externe droite) :** symétrique de la précédente, conserve tous les tuples de la table de droite.
- **FULL OUTER JOIN (jointure externe complète) :** conserve l'ensemble des tuples des deux tables, appariés lorsque c'est possible, complétés de NULL sinon.
- **CROSS JOIN (produit cartésien) :** combine chaque tuple de la première table avec chaque tuple de la seconde, sans aucune condition de filtrage (à utiliser avec précaution, le nombre de résultats croît rapidement : le résultat compte n × m lignes).
- **Auto-jointure (self join) :** une table est jointe à elle-même, typiquement pour exploiter une relation hiérarchique interne (par exemple un employé et son responsable, tous deux stockés dans la table Employe).

Exemple — jointure externe gauche — tous les employés, y compris ceux sans département

```
SELECT e.nom, d.nom_departement
FROM Employe e
LEFT JOIN Departement d ON e.id_dept = d.id_dept;
```

Résultat retourné :

nom	nom_departement
Amel	Ventes
Bilal	Ventes
Camille	Ventes
Driss	Ventes
Emna	Ventes
Farid	Ventes
Ghali	Marketing
Hana	Marketing
Imad	Informatique
Jana	NULL

Le mot-clé LEFT JOIN conserve toutes les lignes de la table à sa gauche (Employe), même lorsqu'aucune ligne de Departement ne correspond. C'est le cas de Jana, dont id_dept vaut NULL : elle apparaît malgré tout dans le résultat, avec nom_departement à NULL. Avec un INNER JOIN à la place, cette ligne aurait simplement été absente du résultat (9 lignes au lieu de 10).

Exemple — auto-jointure — chaque employé avec le nom de son responsable

```
SELECT e.nom AS employe, r.nom AS responsable
FROM Employe e
LEFT JOIN Employe r ON e.id_responsable = r.id_emp;
```

Résultat retourné :

employe	responsable
Amel	NULL
Bilal	Amel
Camille	Amel
Driss	Amel
Emna	Amel
Farid	Amel
Ghali	NULL
Hana	Ghali
Imad	NULL
Jana	NULL

La table Employe est ici utilisée deux fois dans la clause FROM, sous deux alias différents (e et r) : e représente l'employé, r représente son responsable. La jointure relie e.id_responsable à r.id_emp. Les employés qui n'ont pas de responsable (Amel, Ghali, Imad — tous trois managers) obtiennent NULL dans la colonne responsable grâce au LEFT JOIN.

2.1.2 Sous-requêtes

Une sous-requête est une requête SQL imbriquée à l'intérieur d'une autre requête, dans la clause SELECT, FROM ou WHERE. On distingue :

- **la sous-requête non corrélée,** évaluée une seule fois indépendamment de la requête externe ;
- **la sous-requête corrélée,** qui référence une colonne de la requête externe et doit donc être réévaluée pour chaque ligne candidate de celle-ci — ce qui la rend potentiellement coûteuse en performance sur de gros volumes.

Exemple — sous-requête non corrélée dans la clause WHERE — employés mieux payés que la moyenne

```
SELECT nom FROM Employe
WHERE salaire > (SELECT AVG(salaire) FROM Employe);
```

Résultat retourné :

nom
Amel
Ghali
Imad

La sous-requête (SELECT AVG(salaire) FROM Employe) est évaluée une seule fois : elle calcule la moyenne des 10 salaires du tableau, soit 41 500. La requête externe compare ensuite chaque salaire individuel à cette valeur fixe. Seuls Amel (45 000), Ghali (52 000) et Imad (60 000) dépassent ce seuil.

Les opérateurs IN, EXISTS, ANY (ou SOME) et ALL permettent de comparer une valeur, ou l'existence de tuples, au résultat d'une sous-requête.

Exemple — sous-requête corrélée avec EXISTS — responsables d'un projet à gros budget

```
SELECT nom FROM Employe e
WHERE EXISTS (
```

```
  SELECT 1 FROM Projet p
  WHERE p.id_resp = e.id_emp AND p.budget > 100000
);
```

Résultat retourné :

nom
Amel
Camille

Contrairement à l'exemple précédent, cette sous-requête référence e.id_emp, une colonne de la requête externe : elle est donc réévaluée pour chaque employé. EXISTS ne teste que la présence d'au moins une ligne correspondante (peu importe combien), ce qui est en général plus efficace qu'un COUNT(*) > 0. En parcourant la table Projet du §2.0, Camille (id_emp 1003) est responsable du projet 1 (budget 150 000 > 100 000) et Amel (id_emp 1001) du projet 3 (budget 120 000 > 100 000) : ces deux employés sont donc retenus, tandis qu'Emna, responsable d'un projet de seulement 80 000, est exclue.

2.1.3 Fonctions d'agrégation, GROUP BY et HAVING

Les fonctions COUNT, SUM, AVG, MIN et MAX permettent de calculer des agrégats sur des ensembles de lignes définis par la clause GROUP BY. Chaque fonction résume en une seule valeur un ensemble de lignes : COUNT compte le nombre de lignes, SUM additionne une colonne numérique, AVG calcule la moyenne, MIN et MAX renvoient respectivement la plus petite et la plus grande valeur du groupe.

La clause HAVING filtre les groupes ainsi constitués (après agrégation), à la différence de WHERE, qui filtre les lignes individuelles avant tout regroupement. Une clause WHERE ne peut donc jamais porter sur le résultat d'une fonction d'agrégation (par exemple WHERE COUNT(*) > 5 est invalide) ; HAVING est indispensable pour cela.

Exemple — GROUP BY + HAVING — départements de plus de 5 employés

```
SELECT id_dept, COUNT(*) AS effectif, AVG(salaire) AS salaire_moyen
FROM Employe
GROUP BY id_dept
HAVING COUNT(*) > 5;
```

Résultat retourné :

id_dept	effectif	salaire_moyen
10	6	38666.67

GROUP BY id_dept regroupe d'abord les 10 employés en 4 groupes : dept 10 (6 lignes), dept 20 (2 lignes), dept 30 (1 ligne) et dept NULL (1 ligne, Jana). COUNT(*) et AVG(salaire) sont ensuite calculés pour chaque groupe. La clause HAVING COUNT(*) > 5 n'est satisfaite que par le groupe dept 10, seul groupe à compter plus de 5 employés (6) : c'est le seul qui apparaît dans le résultat final, avec un salaire moyen de 232000 / 6 ≈ 38 666,67.

2.1.4 Fonctions analytiques (fonctions de fenêtrage)

Les fonctions analytiques (ou fonctions de fenêtrage, window functions), introduites par SQL:2003, permettent de calculer un agrégat glissant sur un sous-ensemble de lignes (une fenêtre) sans réduire le nombre de lignes du résultat, contrairement à GROUP BY : chaque ligne du résultat conserve son individualité, enrichie d'une valeur calculée sur sa fenêtre. Elles reposent sur la clause OVER, associée éventuellement à :

- **PARTITION BY** pour restreindre la fenêtre à un sous-groupe (l'équivalent d'un regroupement, mais sans fusion des lignes) ;
- **ORDER BY** pour définir un ordre de calcul au sein de la fenêtre, indispensable pour les fonctions de classement.

Exemple — classement des employés par salaire au sein de chaque département

```
SELECT nom, id_dept, salaire,
  RANK() OVER (PARTITION BY id_dept ORDER BY salaire DESC) AS rang,
  AVG(salaire) OVER (PARTITION BY id_dept) AS moyenne_dept
FROM Employe;
```

Résultat retourné :

nom	id_dept	salaire	rang	moyenne_dept
Amel	10	45000	1	38666.67
Camille	10	41000	2	38666.67
Emna	10	39000	3	38666.67
Bilal	10	38000	4	38666.67
Driss	10	36000	5	38666.67
Farid	10	33000	6	38666.67
Ghali	20	52000	1	46500
Hana	20	41000	2	46500
Imad	30	60000	1	60000
Jana	NULL	30000	1	30000

PARTITION BY id_dept découpe les 10 lignes en fenêtres indépendantes, une par département — mais sans fusionner les lignes comme le ferait GROUP BY : les 10 employés apparaissent toujours individuellement dans le résultat. Au sein de chaque fenêtre, RANK() OVER (... ORDER BY salaire DESC) attribue un rang décroissant (Amel, le mieux payé du département 10, obtient le rang 1), tandis que AVG(salaire) OVER (PARTITION BY id_dept), sans ORDER BY, calcule la moyenne du département entière et répète cette même valeur sur toutes les lignes de la fenêtre.

Parmi les fonctions analytiques les plus utilisées :

- **ROW_NUMBER()** attribue un numéro de ligne strictement croissant au sein de la fenêtre, sans jamais répéter un même numéro même en cas d'ex-æquo.
- **RANK()** attribue un rang qui « saute » une valeur après un ex-æquo (deux lignes à égalité obtiennent le même rang, la ligne suivante saute un rang, par ex. 1, 2, 2, 4).
- **DENSE_RANK()** attribue un rang sans « trou » après un ex-æquo (1, 2, 2, 3).
- **LAG() et LEAD()** permettent d'accéder à la valeur d'une ligne précédente (LAG) ou suivante (LEAD) dans l'ordre défini par ORDER BY, très utiles pour des calculs de variation d'une période à l'autre sans recourir à une auto-jointure.

Exemple — LAG() — variation du montant de chaque vente par rapport à la précédente, pour un même vendeur

```
-- Table Ventes(id_vente, id_vendeur, date_vente, montant)
SELECT id_vente, id_vendeur, date_vente, montant,
  montant - LAG(montant) OVER (PARTITION BY id_vendeur ORDER BY date_vente) AS
variation
FROM Ventes
ORDER BY id_vendeur, date_vente;
```

Résultat retourné :

id_vente	id_vendeur	date_vente	montant	variation
1	501	2025-01-05	1000	NULL
2	501	2025-01-12	1500	500
3	501	2025-01-20	1200	-300
4	502	2025-01-08	800	NULL
5	502	2025-01-15	950	150

Pour chaque vente, LAG(montant) OVER (PARTITION BY id_vendeur ORDER BY date_vente) va chercher le montant de la vente immédiatement antérieure du même vendeur. La toute première vente de chaque vendeur (id_vente 1 pour le vendeur 501, id_vente 4 pour le vendeur 502) n'a pas de ligne précédente au sein de sa fenêtre : LAG() renvoie donc NULL, et la variation calculée (montant − NULL) vaut également NULL. Pour la

vente 2, la variation est 1500 − 1000 = 500 ; pour la vente 3, elle devient négative : 1200 − 1500 = −300, traduisant une baisse par rapport à la vente précédente du même vendeur.

Exemple — cumul progressif (running total) des ventes mensuelles

```
-- Table VentesMensuelles(mois, montant)
SELECT mois, montant,
  SUM(montant) OVER (ORDER BY mois
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumul
FROM VentesMensuelles;
```

Résultat retourné :

mois	montant	cumul
2025-01	5000	5000
2025-02	3000	8000
2025-03	4000	12000

La clause ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW définit une fenêtre qui s'étend, pour chaque ligne, de la toute première ligne (UNBOUNDED PRECEDING) jusqu'à la ligne courante (CURRENT ROW) selon l'ordre défini par ORDER BY mois. SUM(montant) est donc recalculée à chaque ligne sur un ensemble qui grandit progressivement : 5000, puis 5000+3000=8000, puis 8000+4000=12000. C'est le mécanisme du cumul progressif, très utilisé pour les tableaux de bord (chiffre d'affaires cumulé, par exemple).

2.1.5 GROUPING SETS, CUBE et ROLLUP

Les extensions GROUPING SETS, CUBE et ROLLUP permettent de calculer en une seule requête plusieurs niveaux d'agrégation combinés (par exemple, le total par département, le total par année, et le total général), évitant l'écriture et l'exécution de plusieurs requêtes UNION distinctes.

- **ROLLUP(a, b)** calcule une hiérarchie de sous-totaux emboîtés, dans l'ordre des colonnes indiquées : le détail par (a, b), puis le sous-total par a seul, puis le total général.
- **CUBE(a, b)** va plus loin que ROLLUP : il calcule toutes les combinaisons possibles de sous-totaux, y compris le sous-total par b seul (que ROLLUP ne calcule pas).
- **GROUPING SETS(...)** permet de spécifier explicitement, à la carte, exactement quels regroupements calculer, sans être limité à la hiérarchie imposée par ROLLUP ou à l'exhaustivité de CUBE.

Exemple — ROLLUP — sous-totaux par département, par année, et total général

```
-- Table Ventes(id_dept, annee, montant)
SELECT id_dept, annee, SUM(montant) AS total
FROM Ventes
GROUP BY ROLLUP(id_dept, annee);
```

Résultat retourné :

id_dept	annee	total
10	2024	120000
10	2025	150000
10	NULL	270000 (sous-total dept 10)
20	2024	90000
20	2025	95000
20	NULL	185000 (sous-total dept 20)
NULL	NULL	455000 (total général)

En partant d'une table Ventes à 4 lignes (120000+150000+90000+95000=455000 au total), ROLLUP(id_dept, annee) ajoute automatiquement trois lignes de sous-total à celles du détail : une ligne par département (annee = NULL, qui additionne toutes les années de ce département), et une ligne finale tous départements et années

confondus (id_dept = NULL et annee = NULL). Le NULL dans ces lignes ne signifie donc pas « valeur inconnue » mais « toutes les valeurs de cette colonne agrégées ensemble ».

Exemple — GROUPING SETS — sous-totaux par département SEUL et par année SEULE (sans détail croisé)

```
SELECT id_dept, annee, SUM(montant) AS total
FROM Ventes
GROUP BY GROUPING SETS((id_dept), (annee));
```

Résultat retourné :

id_dept	annee	total
10	NULL	270000
20	NULL	185000
NULL	2024	210000
NULL	2025	245000

Contrairement à ROLLUP, GROUPING SETS((id_dept), (annee)) ne calcule que les deux regroupements explicitement listés : par id_dept seul, et par annee seul. Il ne produit ni le détail croisé (id_dept, annee), ni le total général, ni même les deux à la fois — seulement les 4 lignes demandées. C'est cette liberté de spécification qui distingue GROUPING SETS de ROLLUP (hiérarchie imposée) et de CUBE (toutes les combinaisons).

2.1.6 Vues et vues matérialisées

Une vue est une requête stockée qui se comporte, du point de vue de l'utilisateur, comme une table virtuelle : chaque fois qu'elle est interrogée, sa requête sous-jacente est réexécutée sur les données à jour. Elle simplifie l'écriture de requêtes complexes récurrentes, restreint l'accès de certains utilisateurs à un sous-ensemble de colonnes ou de lignes d'une table (mécanisme de sécurité par restriction), et assure une indépendance logique vis-à-vis des évolutions du schéma physique sous-jacent.

Une vue matérialisée, à la différence d'une vue classique, stocke physiquement le résultat de la requête au moment de sa création ou de son dernier rafraîchissement. Elle offre un gain de performance important pour des agrégats coûteux et fréquemment consultés, au prix d'un risque de désynchronisation temporaire avec les données sources tant qu'elle n'a pas été rafraîchie (stratégies de rafraîchissement : à la demande, périodique, ou en temps réel via des déclencheurs).

Exemple — vue matérialisée — total des ventes par département et par mois

```
-- Table Ventes(id_vente, id_dept, date_vente, montant)
CREATE MATERIALIZED VIEW mv_ventes_mensuelles AS
SELECT id_dept, DATE_TRUNC('month', date_vente) AS mois, SUM(montant) AS total
FROM Ventes
GROUP BY id_dept, DATE_TRUNC('month', date_vente);

REFRESH MATERIALIZED VIEW mv_ventes_mensuelles;
SELECT * FROM mv_ventes_mensuelles;
```

Résultat retourné :

id_dept	mois	total
10	2025-01-01	8000
10	2025-02-01	4000
20	2025-01-01	7000

DATE_TRUNC('month', date_vente) tronque chaque date au premier jour du mois, ce qui permet de regrouper toutes les ventes d'un même mois ensemble. En supposant une table Ventes contenant deux ventes du département 10 en janvier (5000 et 3000, soit un total de 8000), une vente du département 10 en février (4000), et une vente du département 20 en janvier (7000), le résultat matérialisé contient exactement ces trois lignes agrégées. Contrairement à une vue classique, ce résultat est stocké physiquement : il ne sera à jour que jusqu'au prochain REFRESH MATERIALIZED VIEW, même si de nouvelles ventes sont insérées entre-temps.

2.1.7 Fonctions et procédures stockées

Une procédure stockée est un ensemble d'instructions SQL et procédurales (PL/SQL, T-SQL, PL/pgSQL selon le SGBD) compilé et stocké au sein même du SGBD, invocable par l'instruction CALL. Une fonction stockée, à la différence d'une procédure, retourne obligatoirement une valeur et peut ainsi être directement utilisée à l'intérieur d'une expression SQL (par exemple dans une clause SELECT ou WHERE).

Le recours aux procédures et fonctions stockées présente plusieurs avantages :

- Réduction du trafic réseau : un seul appel remplace plusieurs allers-retours de requêtes.
- Centralisation de la logique métier critique au plus près des données.
- Amélioration de la sécurité : les utilisateurs applicatifs peuvent se voir accorder uniquement le droit d'exécuter la procédure, sans accès direct aux tables sous-jacentes.
- Réutilisation du code entre plusieurs applications clientes.

```
CREATE OR REPLACE PROCEDURE augmenter_salaire(
  p_id_emp IN NUMBER, p_taux IN NUMBER)
IS
BEGIN
  UPDATE Employe
  SET salaire = salaire * (1 + p_taux)
  WHERE id_emp = p_id_emp;
  COMMIT;
END;
```

Exemple — appel de la procédure augmenter_salaire — augmenter le salaire de Bilal de 5 %

```
CALL augmenter_salaire(1002, 0.05);
SELECT id_emp, nom, salaire FROM Employe WHERE id_emp = 1002;
```

Résultat retourné :

id_emp	nom	salaire avant	salaire après
1002	Bilal	38000	39900

La procédure reçoit p_id_emp=1002 et p_taux=0.05, et exécute UPDATE Employe SET salaire = salaire * (1+0.05) WHERE id_emp = 1002. Le salaire de Bilal, initialement 38000, est multiplié par 1,05, ce qui donne 39900. Comme il s'agit d'une procédure (pas d'une fonction), elle ne renvoie aucune valeur : son effet se constate uniquement en relisant la table Employe après l'appel.

```
CREATE OR REPLACE FUNCTION calculer_prime(p_salaire NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN CASE WHEN p_salaire > 50000 THEN p_salaire * 0.05
              ELSE p_salaire * 0.10 END;
END;
```

Exemple — utilisation de la fonction calculer_prime dans une clause SELECT

```
SELECT nom, salaire, calculer_prime(salaire) AS prime
FROM Employe;
```

Résultat retourné :

nom	salaire	prime
Amel	45000	4500
Bilal	38000	3800
Camille	41000	4100
Driss	36000	3600
Emna	39000	3900
Farid	33000	3300

nom	salaire	prime
Ghali	52000	2600
Hana	41000	4100
Imad	60000	3000
Jana	30000	3000

Une fonction stockée, contrairement à une procédure, retourne une valeur et peut donc être appelée directement dans la liste des colonnes d'un SELECT, exactement comme une fonction native. Pour chaque ligne, calculer_prime applique la règle : 5 % du salaire s'il dépasse 50000, sinon 10 %. Ghali (52000 > 50000) et Imad (60000 > 50000) obtiennent donc un taux de 5 % (respectivement 2600 et 3000), tandis que tous les autres employés, en dessous du seuil, obtiennent 10 % de leur salaire.

Exemple — utilisation de calculer_prime dans une clause WHERE (filtrage)

```
SELECT nom FROM Employe
WHERE calculer_prime(salaire) > 3000;
```

Résultat retourné :

nom
Amel
Bilal
Camille
Driss
Emna
Farid
Hana

SQL évalue ici calculer_prime(salaire) pour chaque employé avant de comparer le résultat à 3000. D'après le tableau précédent, seuls Ghali (2600) et Imad (3000, non strictement supérieur à 3000) ne satisfont pas la condition : les 7 autres employés, dont la prime calculée dépasse strictement 3000, sont retenus.

Les curseurs explicites permettent de parcourir ligne à ligne le résultat d'une requête à l'intérieur d'un bloc procédural, lorsqu'un traitement séquentiel spécifique doit être appliqué à chaque ligne (traitement qu'il n'est pas possible d'exprimer en SQL déclaratif pur).

```
DECLARE
  CURSOR c_employes IS SELECT id_emp, salaire FROM Employe WHERE id_dept = 10;
  v_id NUMBER; v_sal NUMBER;
BEGIN
  OPEN c_employes;
  LOOP
    FETCH c_employes INTO v_id, v_sal;
    EXIT WHEN c_employes%NOTFOUND;
    UPDATE Employe SET prime = calculer_prime(v_sal) WHERE id_emp = v_id;
  END LOOP;
  CLOSE c_employes;
  COMMIT;
END;
```

Résultat retourné :

id_emp	nom	salaire	prime (colonne mise à jour)
1001	Amel	45000	4500
1002	Bilal	38000	3800
1003	Camille	41000	4100

id_emp	nom	salaire	prime (colonne mise à jour)
1004	Driss	36000	3600
1005	Emna	39000	3900
1006	Farid	33000	3300

Le curseur c_employes sélectionne uniquement les 6 employés du département 10. La boucle FETCH ... EXIT WHEN %NOTFOUND parcourt ces lignes une par une (v_id, v_sal), et appelle calculer_prime(v_sal) pour mettre à jour la colonne prime de chaque employé individuellement — un traitement que GROUP BY ou une simple UPDATE en une seule instruction ne permettraient pas d'exprimer aussi finement lorsqu'une logique ligne par ligne plus complexe est requise. Les valeurs obtenues sont ici identiques à celles calculées directement par la fonction, puisque tous ces salaires restent sous le seuil de 50000.

2.1.8 Exemples d'appel et d'utilisation pratique

Une fois créée, une procédure stockée s'invoque avec l'instruction CALL (norme SQL) ou EXEC (syntaxe raccourcie propre à certains SGBD comme SQL Server ou Oracle), en lui passant les paramètres attendus dans l'ordre de sa déclaration, ou nommément pour plus de lisibilité.

```
-- Appel simple, paramètres positionnels
CALL augmenter_salaire(1002, 0.05);

-- Appel avec paramètres nommés (plus lisible, ordre indifférent)
CALL augmenter_salaire(p_id_emp => 1002, p_taux => 0.05);
```

Les deux appels ci-dessus sont strictement équivalents et produisent le même résultat déjà illustré plus haut (salaire de Bilal porté à 39900) : seule la syntaxe d'invocation change selon le contexte (SQL pur, SGBD propriétaire,....).

L'exemple ci-dessous illustre un cas complet et représentatif : une procédure de virement bancaire entre deux comptes, combinant paramètres, contrôle métier, gestion d'exception et validation transactionnelle explicite, telle qu'on la rencontre couramment en environnement de production.

```
CREATE OR REPLACE PROCEDURE virement(
  p_compte_source IN NUMBER, p_compte_dest IN NUMBER, p_montant IN NUMBER)
IS
  v_solde NUMBER;
BEGIN
  -- Verrouille la ligne source pour éviter une lecture concurrente incohérente
  SELECT solde INTO v_solde FROM Compte
  WHERE id_compte = p_compte_source FOR UPDATE;

  IF v_solde < p_montant THEN
    RAISE_APPLICATION_ERROR(-20002, 'Solde insuffisant');
  END IF;

  UPDATE Compte SET solde = solde - p_montant WHERE id_compte = p_compte_source;
  UPDATE Compte SET solde = solde + p_montant WHERE id_compte = p_compte_dest;
  COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
```

Exemple — appel de virement — table Compte(id_compte, solde) avec 101 → solde 5000 et 202 → solde 1000

```
CALL virement(101, 202, 1500);
SELECT id_compte, solde FROM Compte WHERE id_compte IN (101, 202);
```

Résultat retourné :

id_compte	solde avant	solde après
101	5000	3500
202	1000	2500

v_solde reçoit d'abord le solde du compte source (5000). Comme 5000 ≥ 1500, aucune exception n'est levée : les deux UPDATE s'exécutent, débitant 1500 du compte 101 (5000 − 1500 = 3500) et créditant 1500 au compte 202 (1000 + 1500 = 2500). Le COMMIT final valide définitivement les deux écritures ensemble. Si le solde source avait été inférieur à 1500, RAISE_APPLICATION_ERROR aurait interrompu la procédure avant toute écriture, et le bloc EXCEPTION aurait déclenché un ROLLBACK, annulant tout effet partiel.

> *Remarque pratique : l'instruction FOR UPDATE utilisée ci-dessus pose un verrou exclusif sur la ligne lue, empêchant une autre transaction de modifier ce même compte avant la fin du virement*

2.2 Déclencheurs, gestion des erreurs

2.2.1 Les déclencheurs (triggers)

Un déclencheur (trigger) est un bloc de code exécuté automatiquement par le SGBD en réaction à un événement survenant sur une table ou une vue : insertion (INSERT), modification (UPDATE) ou suppression (DELETE).

- **BEFORE** (avant l'événement) : permet de valider ou corriger les données avant leur écriture effective, ou de bloquer l'opération en levant une exception.
- **AFTER** (après l'événement) : permet de réagir à un état déjà consolidé, par exemple pour alimenter un journal d'audit.
- **FOR EACH ROW** le trigger s'exécute une fois pour chaque tuple affecté par l'instruction déclenchante.
- **FOR EACH STATEMENT** le trigger s'exécute une seule fois, quel que soit le nombre de lignes concernées.

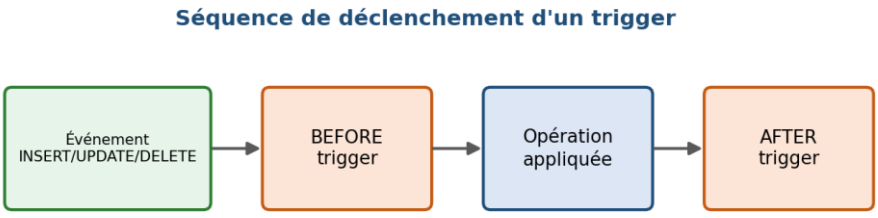


Figure 2.2 — Séquence de déclenchement d'un trigger BEFORE / AFTER

```
CREATE OR REPLACE TRIGGER trg_audit_salaire
AFTER UPDATE OF salaire ON Employe
FOR EACH ROW
BEGIN
  INSERT INTO Historique_Salaire
    (id_emp, ancien_salaire, nouveau_salaire, date_modif)
  VALUES
    (:OLD.id_emp, :OLD.salaire, :NEW.salaire, SYSDATE);
END;
```

Exemple — déclenchement de trg_audit_salaire

```
UPDATE Employe SET salaire = 47000 WHERE id_emp = 1002; -- Bilal, salaire initial
38000
```

Résultat retourné :

id_emp	ancien_salaire	nouveau_salaire	date_modif
1002	38000	47000	(date/heure de l'UPDATE)

Le trigger n'est pas appelé explicitement : c'est l'UPDATE lui-même qui le déclenche automatiquement, après (AFTER) que la modification a été appliquée. Les pseudo-variables :OLD et :NEW donnent accès respectivement à la valeur avant modification (38000) et après modification (47000) de la ligne concernée ; une ligne d'audit est insérée automatiquement dans Historique_Salaire, sans aucune action supplémentaire requise de la part de l'utilisateur ayant exécuté l'UPDATE.

Les triggers INSTEAD OF, spécifiques aux vues (en particulier celles portant sur plusieurs tables jointes, non modifiables directement), permettent de définir explicitement comment traduire une opération de modification exprimée sur la vue en opérations réelles sur les tables sous-jacentes.

L'usage des triggers doit rester mesuré : des triggers en cascade (un trigger déclenchant à son tour une modification qui active un autre trigger) nuisent gravement à la lisibilité et à la maintenabilité du code, rendent le comportement du système difficile à prévoir, et peuvent dégrader sensiblement les performances. Une documentation rigoureuse de chaque trigger actif sur une table est indispensable.

2.2.2 Exemples d'utilisation courante des triggers

Au-delà de l'exemple d'audit présenté ci-dessus, les triggers répondent à un ensemble de besoins récurrents qu'il est utile d'illustrer précisément, car ce sont les cas d'usage les plus fréquemment rencontrés en pratique.

a) Validation et normalisation de données avant écriture (BEFORE INSERT)

Un trigger BEFORE INSERT permet de contrôler ou corriger une donnée avant son enregistrement effectif, par exemple pour refuser une valeur incohérente ou normaliser un format de saisie.

```
CREATE OR REPLACE TRIGGER trg_valider_employe
BEFORE INSERT OR UPDATE ON Employe
FOR EACH ROW
BEGIN
  IF :NEW.salaire <= 0 THEN
    RAISE_APPLICATION_ERROR(-20003, 'Le salaire doit être positif');
  END IF;
  :NEW.email := LOWER(:NEW.email);   -- normalisation de la casse
  :NEW.date_maj := SYSDATE;          -- horodatage automatique
END;
```

Résultat retourné :

Tentative d'insertion	Effet du trigger
INSERT ... VALUES (1011,'Karim','KARIM@CORP.COM',42000,10,1001,'EMPLOYE');	Insertion acceptée. email stocké en minuscules : 'karim@corp.com'. date_maj fixée automatiquement à l'instant courant.
INSERT ... VALUES (1012,'Lina','lina@corp.com',-500 ,10,1001,'EMPLOYE');	Insertion REFUSÉE : erreur -20003 « Le salaire doit être positif » levée avant toute écriture.

Le trigger s'exécute avant (BEFORE) que la ligne ne soit réellement écrite : il peut donc encore modifier :NEW (la future ligne) ou empêcher totalement l'opération en levant une exception, ce qu'un trigger AFTER ne pourrait plus faire puisque l'écriture aurait déjà eu lieu.

b) Génération automatique d'un identifiant (BEFORE INSERT + séquence)

Avant l'apparition des colonnes auto-incrémentées natives (IDENTITY, SERIAL), les triggers étaient le mécanisme standard pour générer automatiquement une clé primaire à partir d'une séquence ; ce motif reste très répandu sous Oracle.

```
CREATE SEQUENCE seq_client START WITH 1 INCREMENT BY 1;
```

```
CREATE OR REPLACE TRIGGER trg_id_client
BEFORE INSERT ON Client
FOR EACH ROW
WHEN (NEW.id_client IS NULL)
BEGIN
  :NEW.id_client := seq_client.NEXTVAL;
END;
```

Exemple — insertion d'un nouveau client sans préciser d'identifiant

```
INSERT INTO Client (nom) VALUES ('Nadia');
SELECT * FROM Client WHERE nom = 'Nadia';
```

Résultat retourné :

id_client	nom
1	Nadia

La clause WHEN (NEW.id_client IS NULL) restreint le trigger au cas où l'identifiant n'a pas été fourni explicitement. Ici, l'INSERT n'indique aucune valeur pour id_client : le trigger intercepte l'insertion avant son écriture et appelle seq_client.NEXTVAL, qui renvoie 1 (première valeur de la séquence). La ligne finalement écrite contient donc id_client=1, sans que l'utilisateur ait eu à s'en soucier.

c) Maintien d'une donnée dénormalisée (AFTER INSERT)

Un trigger peut maintenir automatiquement à jour un agrégat dénormalisé (ici, le nombre total de commandes d'un client), évitant de le recalculer à chaque lecture par une requête COUNT coûteuse.

```
CREATE OR REPLACE TRIGGER trg_maj_nb_commandes
AFTER INSERT ON Commande
FOR EACH ROW
BEGIN
  UPDATE Client
  SET nb_commandes = nb_commandes + 1
  WHERE id_client = :NEW.id_client;
END;
```

Exemple — insertion d'une nouvelle commande pour le client 1 (Nadia, nb_commandes = 3 avant insertion)

```
INSERT INTO Commande (id_commande, id_client) VALUES (501, 1);
SELECT id_client, nom, nb_commandes FROM Client WHERE id_client = 1;
```

Résultat retourné :

id_client	nom	nb_commandes avant	nb_commandes après
1	Nadia	3	4

L'INSERT dans Commande déclenche automatiquement, après son exécution, la mise à jour de la ligne correspondante dans Client : nb_commandes passe de 3 à 4. Cette colonne dénormalisée évite ainsi d'exécuter, à chaque consultation de la fiche client, une requête SELECT COUNT(*) FROM Commande WHERE id_client=1 potentiellement coûteuse sur une grande table de commandes.

d) Rendre une vue jointe modifiable (INSTEAD OF)

Une vue combinant plusieurs tables par jointure n'est généralement pas modifiable directement par le SGBD. Un trigger INSTEAD OF intercepte l'ordre de modification exprimé sur la vue et le traduit explicitement en opérations sur les tables réelles sous-jacentes.

```
CREATE VIEW Vue_Employe_Dept AS
SELECT e.id_emp, e.nom, e.salaire, d.nom_departement
FROM Employe e JOIN Departement d ON e.id_dept = d.id_dept;

CREATE OR REPLACE TRIGGER trg_maj_vue_employe
INSTEAD OF UPDATE ON Vue_Employe_Dept
FOR EACH ROW
```

```
BEGIN
  UPDATE Employe
  SET nom = :NEW.nom, salaire = :NEW.salaire
  WHERE id_emp = :NEW.id_emp;
END;
```

Exemple — modification de la vue jointe (Bilal, salaire initial 38000)

```
UPDATE Vue_Employe_Dept SET salaire = 44000 WHERE id_emp = 1002;
SELECT id_emp, salaire FROM Employe WHERE id_emp = 1002;
```

Résultat retourné :

id_emp	salaire avant	salaire après (table Employe réelle)
1002	38000	44000

Sans le trigger INSTEAD OF, l'UPDATE sur Vue_Employe_Dept échouerait, car la vue résulte d'une jointure entre deux tables et le SGBD ne sait pas, par défaut, sur laquelle des deux appliquer la modification. Le trigger INSTEAD OF remplace entièrement l'opération demandée sur la vue par l'instruction qu'il contient : ici, un UPDATE explicite sur la vraie table Employe. Le résultat est bien visible dans Employe (38000 → 44000), alors qu'aucune instruction n'a jamais été exécutée directement sur cette table.

e) Empêcher une opération interdite (BEFORE DELETE)

Un trigger peut également bloquer purement et simplement une opération non autorisée par une règle métier, en levant une exception avant que la suppression ne soit appliquée.

```
CREATE OR REPLACE TRIGGER trg_interdire_suppr_admin
BEFORE DELETE ON Employe
FOR EACH ROW
WHEN (OLD.fonction = 'ADMIN')
BEGIN
  RAISE_APPLICATION_ERROR(-20004,
    'Suppression interdite pour un compte administrateur');
END;
```

Exemple — tentative de suppression d'Amel (id_emp 1001, fonction = ADMIN)

```
DELETE FROM Employe WHERE id_emp = 1001;
```

Résultat retourné :

Résultat
Erreur -20004 : « Suppression interdite pour un compte administrateur ». La ligne d'Amel n'est PAS supprimée.

La clause WHEN (OLD.fonction = 'ADMIN') restreint le déclenchement du trigger aux seules lignes dont la fonction vaut ADMIN : c'est le cas d'Amel dans notre table de base. Le trigger s'exécute avant la suppression effective (BEFORE DELETE) et lève une exception, ce qui interrompt l'instruction DELETE avant qu'elle n'affecte la table. Une suppression visant Bilal (fonction EMPLOYE), en revanche, ne déclencherait pas cette condition et s'exécuterait normalement.

2.2.3 Traitement et gestion des erreurs

Les langages procéduraux offrent des blocs de gestion d'exception (bloc EXCEPTION en PL/SQL, structure TRY...CATCH en T-SQL) permettant d'intercepter une erreur survenue pendant l'exécution — division par zéro, violation de contrainte d'intégrité, absence de résultat pour un SELECT INTO — sans provoquer l'arrêt brutal et incontrôlé de l'application appelante.

```
BEGIN
  UPDATE Compte SET solde = solde - 500 WHERE id_compte = 999;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Compte introuvable');
  END IF;
  COMMIT;
EXCEPTION
```

```
  WHEN NO_DATA_FOUND THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Aucune donnée trouvée');
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
```

Résultat retourné :

Étape	Effet
UPDATE ... WHERE id_compte = 999	Aucune ligne ne correspond (seuls les comptes 101 et 202 existent) : SQL%ROWCOUNT vaut 0.
IF SQL%ROWCOUNT = 0	Condition vraie : RAISE_APPLICATION_ERROR lève l'exception -20001.
Bloc EXCEPTION	L'exception est interceptée par WHEN NO_DATA_FOUND (ou WHEN OTHERS selon le SGBD) : ROLLBACK annule toute modification déjà effectuée, puis le message « Aucune donnée trouvée » est affiché — le bloc se termine proprement, sans plantage brutal.

On observe ici le rôle central de SQL%ROWCOUNT : cette variable système contient le nombre de lignes affectées par la dernière instruction DML. La tester juste après l'UPDATE permet de détecter, de façon fiable, qu'aucun compte 999 n'existe, sans attendre une erreur SGBD moins explicite.

Une bonne pratique de conception consiste à isoler et traiter spécifiquement les erreurs prévisibles (comme NO_DATA_FOUND ci-dessus), à journaliser systématiquement les erreurs imprévues à des fins de diagnostic ultérieur, et à garantir la cohérence transactionnelle en déclenchant un ROLLBACK explicite dès qu'une erreur invalide l'état courant de la transaction.

Les points de sauvegarde (SAVEPOINT) permettent, dans les traitements complexes composés de plusieurs étapes, d'annuler seulement une partie des opérations effectuées plutôt que la totalité de la transaction.

```
-- Compte 101 : solde 5000 ; Compte 202 : solde 150
BEGIN
  UPDATE Compte SET solde = solde - 100 WHERE id_compte = 101;
  SAVEPOINT apres_debit;
  UPDATE Compte SET solde = solde - 200 WHERE id_compte = 202;
  IF (SELECT solde FROM Compte WHERE id_compte = 202) < 0 THEN
    ROLLBACK TO SAVEPOINT apres_debit;
  END IF;
  COMMIT;
END;
```

Résultat retourné :

Étape	Compte 101	Compte 202
État initial	5000	150
Après 1er UPDATE (id_compte=101)	4900	150 (inchangé)
SAVEPOINT apres_debit posé ici	4900	150
Après 2e UPDATE (id_compte=202)	4900	-50 (temporaire, avant test)
ROLLBACK TO SAVEPOINT apres_debit (car -50 < 0)	4900 (conservé)	150 (le 2e UPDATE est annulé)
Après COMMIT final	4900	150

Le premier UPDATE porte le solde du compte 101 à 4900. SAVEPOINT apres_debit marque alors un point de reprise intermédiaire, sans rien valider de façon définitive. Le second UPDATE ferait passer le solde du compte 202 à 150 − 200 = −50, une valeur invalide détectée par le IF. ROLLBACK TO SAVEPOINT apres_debit n'annule que les opérations effectuées après ce point précis (donc uniquement le second UPDATE) : le solde du

compte 202 redevient 150, tandis que le premier UPDATE (déjà passé au moment du SAVEPOINT) reste intact. Le COMMIT final valide donc un débit partiel cohérent : seul le compte 101 a été modifié.

2.5 Transactions avancées : points de sauvegarde et niveaux d'isolation

Au-delà du couple COMMIT/ROLLBACK de base (déjà présenté au chapitre 1) et des points de sauvegarde SAVEPOINT (déjà illustrés en détail au 2.2.3), SQL permet de configurer finement, pour chaque transaction, le niveau de protection accordé face aux transactions concurrentes.

Le niveau d'isolation d'une transaction (fixé par SET TRANSACTION ISOLATION LEVEL) détermine ce compromis, selon un ordre croissant de cohérence — et décroissant de parallélisme possible :

- **READ UNCOMMITTED :** aucune protection, les lectures sales sont possibles (une transaction peut lire une donnée modifiée mais non encore validée par une autre).
- **READ COMMITTED :** empêche les lectures sales ; c'est le niveau par défaut de nombreux SGBD comme PostgreSQL et Oracle.
- **REPEATABLE READ :** empêche en outre les lectures non répétables ; c'est le niveau par défaut de MySQL/InnoDB.
- **SERIALIZABLE :** empêche également les lectures fantômes, garantissant un comportement équivalent à une exécution strictement séquentielle des transactions.

Exemple — fixer explicitement le niveau d'isolation d'une transaction

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
BEGIN;
UPDATE Compte SET solde = solde - 1500 WHERE id_compte = 101;
UPDATE Compte SET solde = solde + 1500 WHERE id_compte = 202;
COMMIT;
```

Résultat retourné :

Cette instruction ne modifie pas ce que fait la transaction, mais la façon dont le SGBD la protège des autres transactions concurrentes. Le chapitre 1 a montré, sur cet exemple même de virement entre comptes, comment un niveau d'isolation insuffisant peut mener à une perte de mise à jour : SERIALIZABLE est le niveau qui offre la garantie la plus forte contre ce type d'anomalie, au prix d'un parallélisme réduit et d'un risque accru d'échecs de transaction à réessayer.